

14/07/2026



Master
Project Management
and Data Science

htw

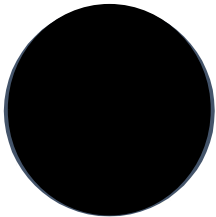
SECOM

Case Study

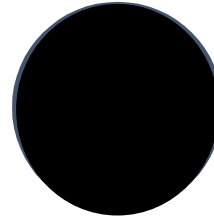
Final Presentation



Team Members



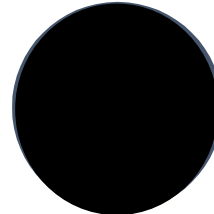
Aya



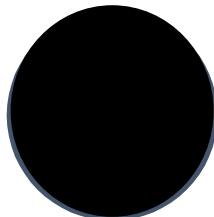
Paula



Runglerdkriangkrai, Thanakorn
S0600588
thanakorn.runglerdkriangkrai@student.htw-berlin.de



Supapat



Titli

Table of Contents

- 1. Project Overview**
- 2. Recap of Previous Work**
- 3. New Advancement**
- 4. How do we find the best model**
- 5. Business Interpretation**
- 6. Limitations, Risks, and Recommendations**

Project Overview

Project Journey and Model Selection

SECOM Defect Detection Project

We analyze semiconductor sensor data to detect defective wafers using machine learning. Focus: building a reliable pipeline that identifies failures early.

Previous Work Completed

Full data cleaning
Imputation & scaling
Baseline Random Forest
Balanced model (Boruta + SMOTE)
GridSearch best model
Result: recall improved but still not optimal

New Work Completed

We tested each preprocessing decision individually:
Outlier handling
Missing value strategies
Feature selection
Balancing methods
Scaling approaches

Final Goal

Identify the **BEST model** that detects defective wafers with the highest recall while maintaining practical precision.

Recap of Previous Work

Our earlier pipeline improved performance but failed to deliver the recall required for reliable defect detection

DATA CLEANING

Flagged missing values to quantify data gaps and prepare for imputation.
Capped outliers using the 3-Sigma rule to stabilize sensor distributions.
Removed features with >60% missing values to reduce noise and improve reliability.
Eliminated constant features that provided no predictive signal.

SCALING AND IMPUTATION

Standardized all sensor features using Standard Scaling to ensure comparable distance calculations.
Applied K-Nearest Neighbors (k = 5) to impute missing values using local similarity patterns.
Preserved all 450 cleaned features after scaling and imputation.

BALANCING

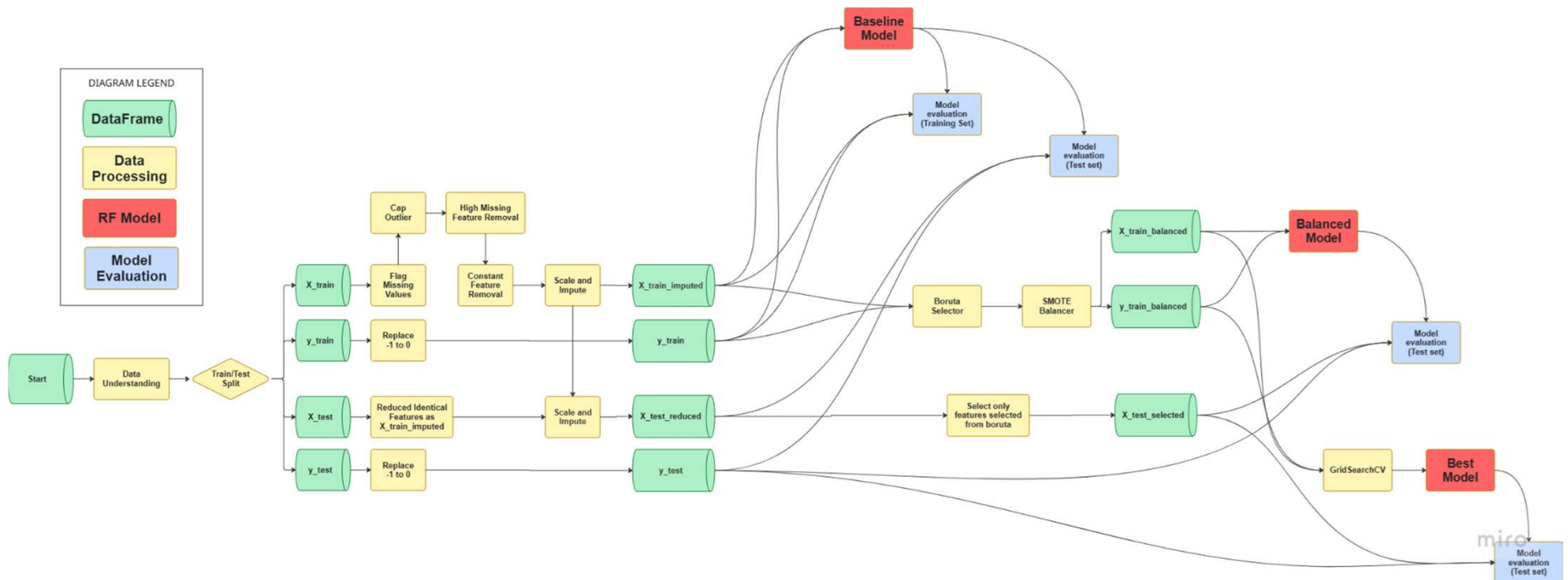
Applied Boruta feature selection using Random Forest to identify healthy sensor features.
Reduced dimensionality from 450 → 12 features, retaining only signals with importance.
Prepared balanced training data by ensuring the minority class receives equal representation.

KEY ISSUE

With GridSearch, recall is 0.62, improved from previous models.
Overall accuracy at 0.51, indicating unstable general performance.
Precision dropped to 0.08, meaning the model raised excessive false alarms.
Outcome: Despite the recall, the model was **not reliable enough** for defect detection in a real manufacturing environment.

Tracking Our End-to-End Pipeline

This diagram maps every step applied to the training and test data



New Advancement Since Last Presentation

We revisited each preprocessing decision to identify which choices truly improve defect detection

Why revisit preprocessing

Our previous best model achieved high recall but suffered from low precision and unstable operational performance.

This suggested that model tuning alone was not enough.

What we suspected

Certain preprocessing choices were limiting model performance, especially:

Outlier Handling, Feature Selection, Scaling and Imputation Method, Balancing Method

These steps shape the data before the model even learns, so poor choices here can reduce performance.

What we tested

We tested different preprocessing choices to isolate the steps that most strongly influence recall and precision.

By testing multiple options we aimed to identify the preprocessing pipeline that delivers the most reliable results

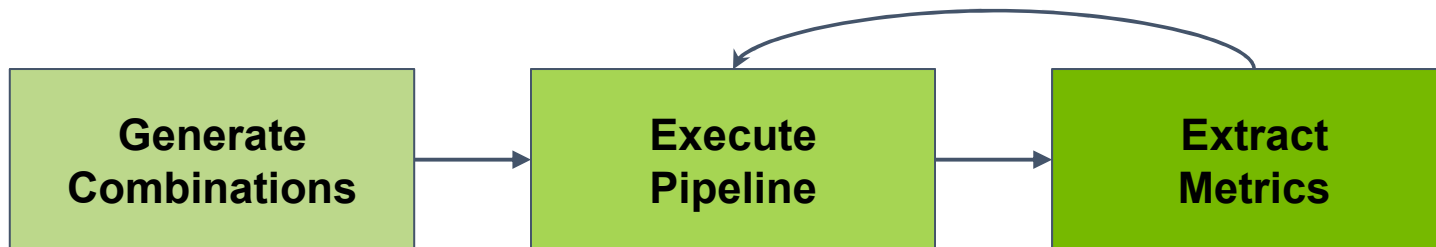
Outlier Handling	cap		remove
Missingness Threshold	0.6		
Scaling Method	standard	minmax	robust
Imputation Parameters	5	7	10
	uniform		distance
Balancing Algorithm	SMOTE	ADASYN	ROSE

Total combinations

108

Testing Each Preprocessing Decisions

We automated all preprocessing combinations to identify the one that delivers the best defect-detection performance



How combinations were created

We generated every possible preprocessing configuration using a Cartesian product.

Code concept:

```
all_combinations =  
product(outliers, missingness,  
scaling, imputation, balancing)
```

Output: [cap, 0.6, standard,...]

How each combinations was run

For each combination, we executed the full training pipeline end-to-end.

Code concept:

```
for combination in all_combinations:  
    processed_data = data_processing(combination)  
    model = train_model(processed_data)  
    evaluation = evaluate_model(model, test_data)  
    results.append(combination, evaluation)
```

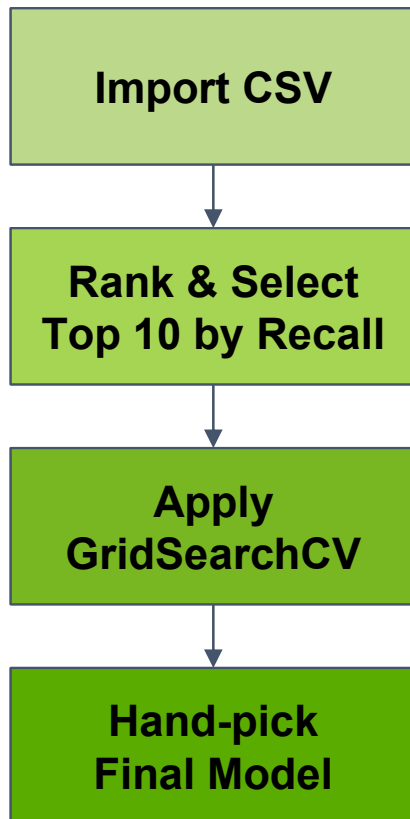
How results were extracted

We stored the metrics for every pipeline configuration to compare performance across all preprocessing choices.

Output: result.csv

How Do We Find the Best Model

A structured, multi-stage process to identify the most reliable defect-detection pipeline



Step 1: Import Evaluation Results in CSV

We loaded the CSV containing all preprocessing combinations and their evaluation metrics.

Step 2: Rank & Select Top 10 by Recall

Recall is the primary metric because defect detection requires catching as many defective wafers as possible.

Step 3: Apply GridSearchCV on the Selected 10

- Run **GridSearchCV** to tune model hyperparameters
- Evaluate tuned models on the test set
- Compare recall, precision, accuracy, and false-positive behavior

Step 4: Hand-pick Final Model

Hand-select the final best model based on:

- highest recall
- acceptable precision
- balanced accuracy

Result After Preprocessing Optimization

The final pipeline selected after ranking, GridSearch refinement, and performance validation

BEST PIPELINE

Outlier Handling	cap
Missingness Threshold	0.6
Scaling Method	robust
Imputation	n_neighbors = 5 weights = uniform
Balancing Algorithm	SMOTE

BEST MODEL

Recall

0.71

Precision

0.11

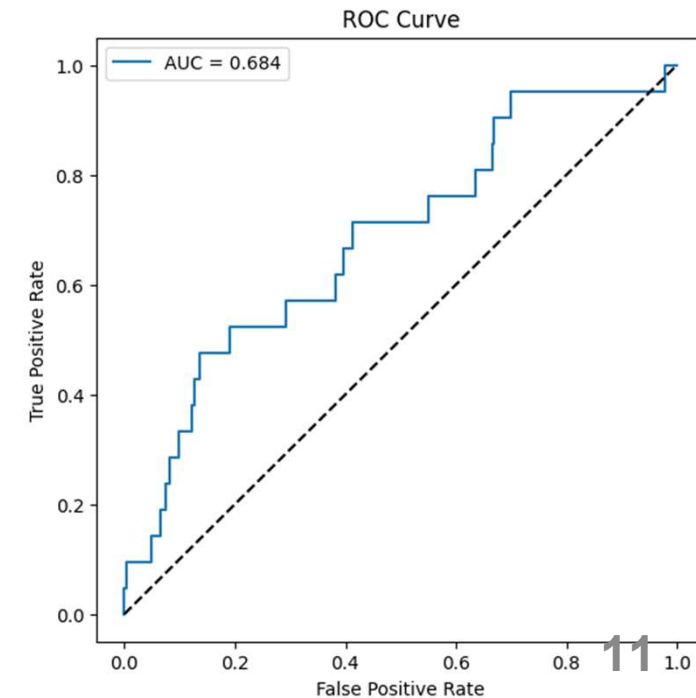
Accuracy

0.58

F1-Score

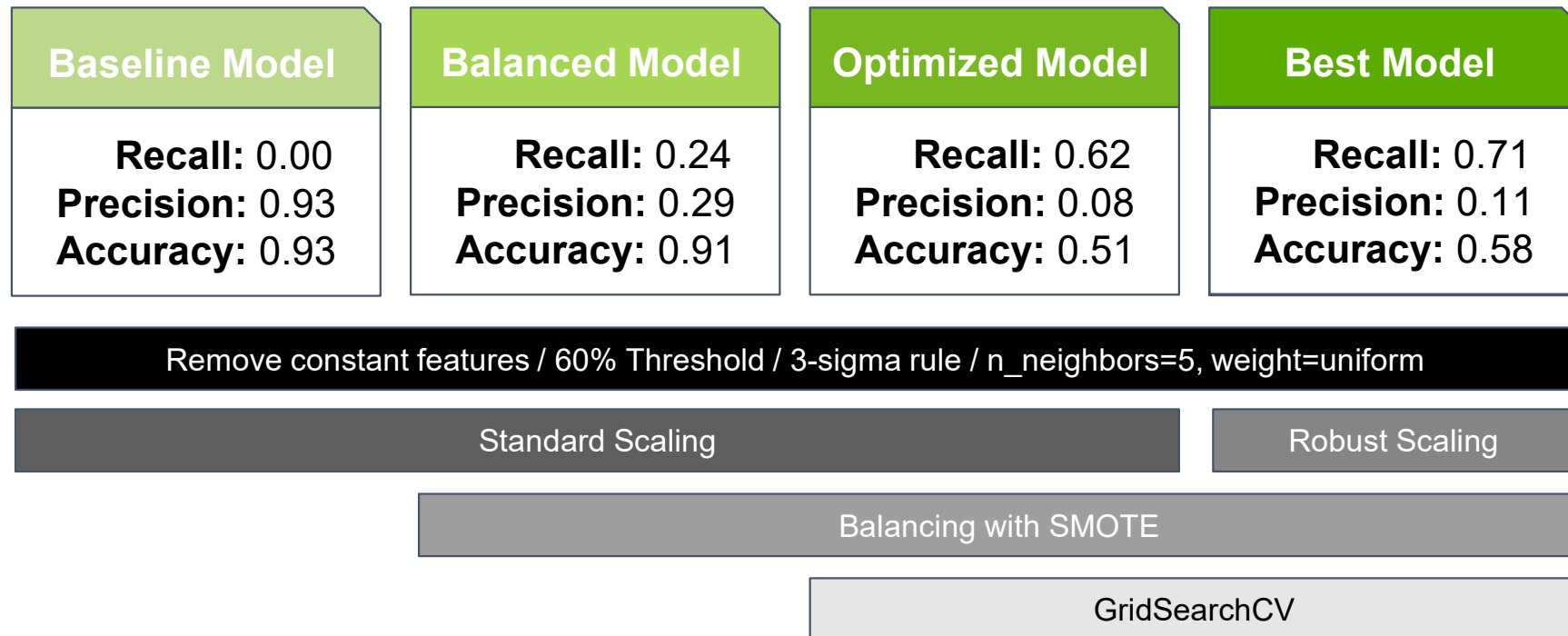
0.18

TN	FP
166	127
FN	TP
6	15



Model Evolution

Comparison of all models and pipelines developed throughout the project



Business Interpretation

What the optimized pipeline means for wafer defect detection and manufacturing risk

What the Final Model Means for Wafer Defect Detection

The final model significantly improves our ability to detect defective wafers early in the manufacturing process.

- Fewer defective wafers escape downstream, reducing scrap, rework, and customer-facing failures.
- Higher confidence in defect screening, enabling more stable yield management.
- Better alignment with manufacturing priorities, where missing a defect is far more costly than flagging a good wafer.

False Negatives vs False Positives

False Negatives (FN) - Missed Defects

Business impact is high and compounding:

- Fewer defective wafers escape downstream, reducing scrap, rework, and customer-facing failures.
- Better alignment with manufacturing priorities, where missing a defect is far more costly than flagging a good wafer.

False Positives (FP) - Good Wafers Flagged as Defective

Business impact is low and contained:

- Additional inspection or re-testing
- Minor throughput slowdown

Why a Recall-Focused Pipeline Is Justified

- **FN cost $\gg$ FP cost**
- Missing a defect is far more expensive than over-flagging a good wafer
- Manufacturing prefers **over-catching** rather than **under-catching**

The final model:

- Maximizes defect capture (high recall)
- Maintains acceptable precision to avoid overwhelming operators
- Provides a stable balance between quality assurance and operational efficiency



Model Limitations



Precision remains moderate

meaning some good wafers will still be flagged for review.



Model performance plateaus with current features

there may be a ceiling unless new features or sensors are added.



Model trained on historical patterns

may not generalize perfectly to new defect types or process drifts.



Higher false positives increase inspection workload

potentially slowing throughput if not managed.



Model drift risk

process changes, tool aging, or recipe updates can degrade performance over time.



Over-reliance on recall

may cause operators to treat alerts as “too frequent,” reducing trust.

Operational Risks

Recommendations



Monitor model drift and set a scheduled retraining cycle

and retrain when process or defect patterns change.



Strengthen alert usability

by improving how model outputs are presented (clearer alert tiers, integrated factory dashboards)



Explore new model families

like xgboost, anomaly detection, deep learning, to push recall–precision performance further.



Vielen Dank!

Sarkar, Titli | El Wardani, Aya | Galvan Rivera, Paula N.
| Wattanadilokkul, Supapat | Runglerdkriangkrai, Thanakorn

www.mpmd.htw-berlin.de